

ผลการรันโค้ด ใช้(Thunder clienในการส่ง API)

4.1 GET ทั้งหมด

The screenshot shows a REST client interface with a dark theme. At the top, the method is set to 'GET' and the URL is 'http://localhost:8080/coffees'. A 'Send' button is on the right. Below the URL bar, there are tabs for 'Query', 'Headers', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Query' tab is selected, showing a table for 'Query Parameters' with columns 'parameter' and 'value'. Below this, the status bar shows 'Status: 200', 'Size: 78 Bytes', and 'Time: 118 ms'. To the right of the status bar is a 'Response' dropdown. The response area displays a JSON array of two coffee objects. A 'Copy' button is located to the right of the JSON text.

```
GET http://localhost:8080/coffees Send
```

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter	value
-----------	-------

Status: 200 Size: 78 Bytes Time: 118 ms Response

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    },
7    {
8      "id": 2,
9      "name": "Latte",
10     "price": 55.0
11  }
```

Copy

4.2 GET ตาม id

The screenshot shows a REST client interface with a dark theme. At the top, the method is set to 'GET' and the URL is 'http://localhost:8080/coffees/1'. A 'Send' button is on the right. Below the URL bar, there are tabs for 'Query', 'Headers', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Query' tab is selected, showing 'Query Parameters' with a table with two columns: 'parameter' and 'value'. Below this, the response status is shown as 'Status: 200', 'Size: 39 Bytes', and 'Time: 17 ms'. A 'Response' dropdown is on the right. The response body is a JSON object displayed with line numbers 1 through 5.

```
1  {
2    "id": 1,
3    "name": "Espresso",
4    "price": 45.0
5  }
```

4.3 POST เพิ่มเมนูใหม่

The screenshot displays a REST client interface with a dark theme. At the top, a dropdown menu is set to 'POST' and the URL is 'http://localhost:8080/coffees'. A 'Send' button is located to the right. Below the URL bar, there are tabs for 'Query', 'Headers 2', 'Auth', 'Body 1', 'Tests', and 'Pre Run'. The 'Body 1' tab is selected, and within it, the 'JSON' format is chosen. The JSON body is as follows:

```
1  {  
2    "name": "Cappuccino",  
3    "price": 60.0  
4  }
```

Below the body editor, the status bar shows 'Status: 201', 'Size: 41 Bytes', and 'Time: 53 ms'. To the right of this is a 'Response' dropdown menu. The response body is displayed below:

```
1  {  
2    "id": 3,  
3    "name": "Cappuccino",  
4    "price": 60.0  
5  }
```

4.4 PUT แก้ว

The screenshot shows a REST client interface with a dark theme. At the top, the method is set to 'PUT' and the URL is 'http://localhost:8080/coffees/2'. A 'Send' button is on the right. Below the URL bar, there are tabs for 'Query', 'Headers 2', 'Auth', 'Body 1', 'Tests', and 'Pre Run'. The 'Body 1' tab is selected, and within it, the 'JSON' format is chosen. The request body contains a JSON object:

```
{  "name": "Latte",  "price": 50.0}
```

. Below the body editor, the status bar shows 'Status: 200', 'Size: 36 Bytes', and 'Time: 11 ms'. A 'Response' dropdown is on the right. The response body is displayed below, showing a JSON object:

```
{  "id": 2,  "name": "Latte",  "price": 50.0}
```

4.5 DELETE ลบ

The screenshot displays a REST client interface with a dark theme. At the top, a request bar shows the method 'DELETE' with a dropdown arrow, the URL 'http://localhost:8080/coffees/3', and a 'Send' button. Below this is a tabbed interface with 'Query', 'Headers 2', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Body' tab is selected, showing a 'JSON' sub-tab with a text area containing the number '1'. At the bottom, a status bar indicates 'Status: 200', 'Size: 0 Bytes', and 'Time: 10 ms'. To the right of the status bar is a 'Response' label with a dropdown arrow. Below the status bar, a large text area shows the response content, which is the number '1'.

4.6 โบนัสเคส (404 Not Found):

The screenshot displays a REST client interface with a dark theme. At the top, a request bar shows the method 'GET' with a dropdown arrow, the URL 'http://localhost:8080/coffees/999', and a blue 'Send' button. Below this is a tabbed interface with 'Query' selected, followed by 'Headers 2', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Query' tab shows a section titled 'Query Parameters' with a table containing one row with an empty checkbox, the text 'parameter', and the text 'value'. At the bottom, a status bar displays 'Status: 404' in red, 'Size: 0 Bytes' in green, 'Time: 9 ms' in green, and 'Response' with a dropdown arrow. Below the status bar, the response body is shown as a single line containing the number '1'.

parameter	value

Status: 404 Size: 0 Bytes Time: 9 ms Response

1

5. Discussion (ตอบในรายงาน)

5.1 ความแตกต่างของ HTTP Methods:

- **GET:** ใช้สำหรับอ่านหรือดึงข้อมูล เช่น ดึงรายการกาแฟทั้งหมด (`/coffees`) หรือดึงตาม ID (`/coffees/1`) โดยไม่เปลี่ยนแปลงข้อมูลบน Server
- **POST:** ใช้สำหรับสร้างข้อมูลใหม่ เช่น การเพิ่มเมนูกาแฟใหม่เข้าในระบบ
- **PUT:** ใช้สำหรับอัปเดตข้อมูลเดิมที่มีอยู่ตาม ID เช่น เปลี่ยนราคาของ Latte
- **DELETE:** ใช้สำหรับลบข้อมูลออกจากระบบตาม ID

5.2 เหตุผลที่ต้องแยก Controller กับ Service:

- เป็นไปตามหลัก **Single Responsibility Principle (SRP)** โดย Controller ทำหน้าที่รับ-ส่ง HTTP Requests/Responses ส่วน Service ทำหน้าที่ประมวลผล Business Logic และจัดการข้อมูล
- **ข้อดีเมื่อโปรแกรมเติบโตขึ้น:** ทำให้สามารถทดสอบโค้ดง่ายขึ้น (Testing), การนำ Logic ไปใช้ซ้ำได้สะดวก (Reusability) และปรับเปลี่ยนแก้ไขได้ง่ายโดยไม่ส่งผลกระทบต่อกัน

5.3 การหายไปของข้อมูลใน Memory และแนวทางแก้ไข:

- ข้อมูลที่เก็บใน `List` จะหายไปเมื่อ แอปพลิเคชันหรือ Server หยุดทำงาน (Stop / Restart) เนื่องจากข้อมูลเก็บไว้ใน RAM เท่านั้น
- **แนวทางแก้ไข:** เชื่อมต่อกับฐานข้อมูลถาวร (Database) เช่น PostgreSQL, MySQL หรือ H2 Database ร่วมกับ Spring Data JPA เพื่อบันทึกข้อมูลลงใน Disk

5.4 หน้าที่ของ Annotations ต่างๆ:

- **@RestController:** ระบุว่าคลาสนี้คือ Controller สำหรับสร้าง REST API ที่คืนค่าเป็น JSON/XML โดยตรง
- **@GetMapping:** แมป HTTP GET Request เข้ากับเมธอดที่กำหนด
- **@PostMapping:** แมป HTTP POST Request เข้ากับเมธอดที่กำหนด
- **@PathVariable:** ดึงค่าจาก Path Parameter ใน URL (เช่น `{id}`) มาเป็นอาร์กิวเมนต์ของเมธอด
- **@RequestBody:** แปลงข้อมูล JSON จาก HTTP Request Body ให้กลายเป็น Java Object โดยอัตโนมัติ